Лабораторная работа 2. Базовый RAG: поиск свидетельств для проверки научных утверждений

Главный вопрос. Помогает ли поиск по коллекции научных аннотаций модели проверять утверждения — и на каком этапе теряется нужная информация?
Что получится. RAG-система на наборе SciFact: плотный поиск по 5183 аннотациям, сборка контекста в заданном бюджете токенов, ответ модели по JSON Schema со ссылкой на документ и предложения. Сравнение с моделью без поиска и с моделью, которой сразу дали эталонное свидетельство; разбор ошибок по этапам; отчёт.
Среда. Каркас ЛР0, сервер модели из ЛР1 или внешний API, библиотека sentence-transformers на CPU.

1. Зачем эта работа

Вот два утверждения из набора SciFact:

- «1/2000 in UK have abnormal PrP positivity»;
- «A total of 1,000 people in the UK are asymptomatic carriers of vCJD infection».

Языковая модель может сразу ответить, верны ли они, но не покажет, на чём основан ответ, и легко ошибётся в узкой медицинской теме. В коллекции статей есть аннотация исследования, где сказано: «Of the 32,441 appendix samples 16 were positive for abnormal PrP, indicating an overall prevalence of 493 per million population». Это предложение **подтверждает** первое утверждение (493 на миллион — примерно 1 на 2000) и **опровергает** второе (при такой частоте носителей в Великобритании десятки тысяч, а не тысяча). Чтобы ответить так, системе нужно найти нужную аннотацию и внимательно прочитать числа.

Генерация, дополненная поиском (retrieval-augmented generation, RAG) [1] соединяет два шага: сначала найти подходящие документы, затем передать их модели вместе с вопросом. Коллекцию можно обновлять, не переобучая модель, а ответ может ссылаться на источник. Но ошибиться можно на каждом шаге: поиск не найдёт нужную аннотацию, нужное предложение не поместится в контекст, модель неправильно прочитает текст или проигнорирует его.

В этой работе вы соберёте базовую RAG-систему, сравните её с моделью без поиска и для каждой ошибки установите, на каком этапе потерялась информация. В ЛР3 вы будете улучшать поиск, поэтому здесь же фиксируется разбиение данных, общее для обеих работ.

Если хотите разобраться в RAG шире, чем требует работа, вот три практических материала. Руководство Mistral AI собирает базовый RAG с нуля на нескольких десятках строк: нарезка документа на фрагменты, эмбеддинги, векторная база, поиск и генерация [8]. Ноутбук русскоязычного практического курса по LLM разбирает те же этапы на LangChain: загрузку и разбиение документов, эмбеддинги через API и модели Hugging Face, векторные хранилища и ретриверы [9]. Ноутбук «Simple Local RAG» строит полностью локальный RAG над PDF-книгой — от чтения файла до ответа модели на собственной GPU [10]. Во всех трёх видно шаг, которого нет в этой работе: длинный документ приходится нарезать на фрагменты (chunking), а в SciFact каждая аннотация — уже готовый фрагмент.

2. Как устроено

2.1. Набор SciFact

SciFact [2] состоит из корпуса и утверждений. В корпусе 5183 аннотации научных статей: у каждой есть `doc_id`, заголовок `title` и текст `abstract` — список предложений. У утверждения есть `id`, текст `claim` и разметка `evidence`: для каждого документа-свидетельства — метка `SUPPORT` или `CONTRADICT` и номера предложений-**обоснований** (rationale), на которых основан вывод [4].

В курсе задача упрощена до выбора одной из трёх меток:

Метка	Когда ставится
`SUPPORT`	В корпусе есть обоснование в пользу утверждения
`CONTRADICT`	В корпусе есть обоснование против утверждения
`NOINFO`	Размеченного обоснования в корпусе нет

Кроме метки система называет документ и номера предложений, на которых основан ответ.

В файле `claims_dev.jsonl` 300 утверждений: 124 с меткой `SUPPORT`, 64 — `CONTRADICT`, 112 — `NOINFO`. У большинства утверждений одно релевантное свидетельство из одного предложения.

2.2. Плотный поиск

Кодировщик (encoder) превращает текст в вектор фиксированной длины так, что тексты, близкие по смыслу, получают близкие векторы [3]. Корпус кодируется один раз; для нового запроса вычисляется только его вектор, и документы ранжируются по косинусной близости:

$$s(q, d) = \frac{e_q^\top e_d}{\|e_q\| \, \|e_d\|}.$$

Для 5183 документов индекс — это просто матрица: у кодировщика с векторами длины 384 она занимает около 7,6 МБ, и поиск перебором по всем строкам выполняется мгновенно. Специальная векторная база здесь не нужна.

У кодировщика есть предел длины входа, и всё, что длиннее, отбрасывается. У `all-MiniLM-L6-v2` предел 256 токенов, и **71% аннотаций SciFact длиннее**: их вектор описывает только начало текста. У `bge-small-en-v1.5` предел 512 токенов, и обрезается 9% аннотаций. Обрезка касается только вектора: модели передаётся полный текст.

2.3. Три списка: найдено, передано, содержит обоснование

Контекст для модели собирается из найденных документов по порядку ранга, целыми предложениями с их исходными номерами `[0]`, `[1]`, … и с ограничением на общее число входных токенов. Поэтому для каждого утверждения важно различать три списка:

- **найдено** — первые k документов поиска;
- **передано** — документы и предложения, которые реально вошли в промпт;
- **содержит обоснование** — в переданном тексте целиком есть хотя бы одно эталонное обоснование.

Если нужный документ найден, но его обоснование не поместилось в бюджет, поиск сработал, а система — нет. Эти три списка и позволяют понять, где потерялась информация.

2.4. Ответ по JSON Schema

Модель отвечает одним JSON-объектом:

```
{"label": "SUPPORT", "doc_id": "13734012", "sentences": [4]}
```

Схема ответа передаётся серверу в параметре `response_format` (structured outputs) [5]. Сервер ограничивает генерацию так, что ответ всегда синтаксически соответствует схеме. Смысл он не проверяет: модель может сослаться на непереданный документ или на предложение, в котором ничего нет. Поэтому ответ всё равно проверяется: метка — одна из трёх, для `SUPPORT` и `CONTRADICT` указаны переданный документ и его предложения, для `NOINFO` — `null` и пустой список.

2.5. Три режима

- **LLM-only** — модель получает только утверждение и может опираться на собственные знания; ссылки не требуются.
- **RAG** — модель получает утверждение и собранный контекст и отвечает только по нему.
- **Oracle** (диагностика) — модель получает одно эталонное обоснование вместо результатов поиска. Режим работает только для `SUPPORT` и `CONTRADICT`: для `NOINFO` эталонного текста нет, а пустой контекст выдал бы метку. Oracle использует разметку, поэтому системой он не является; он отвечает на вопрос, справляется ли модель, когда нужное свидетельство точно перед ней.

Рис. 1 показывает все три пути.

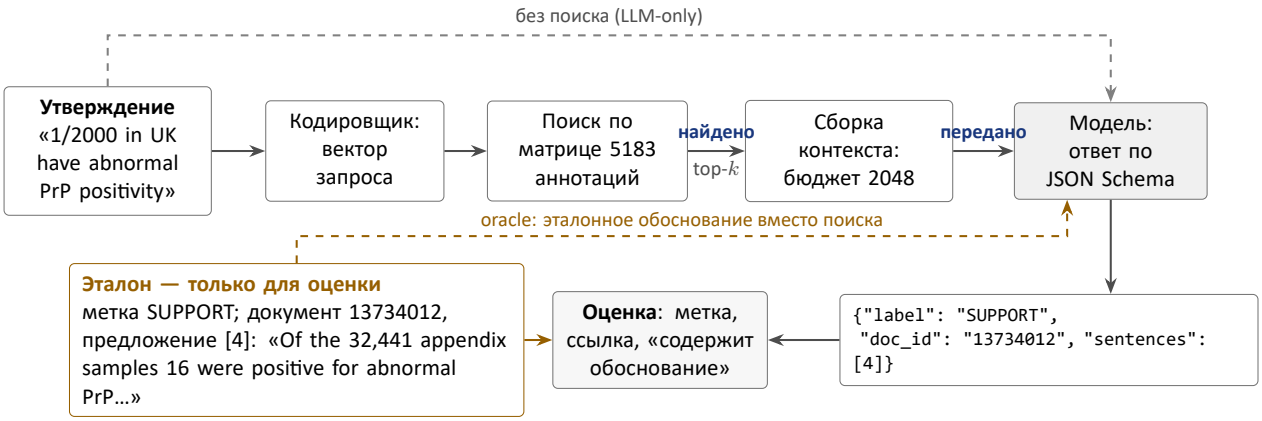


Рис. 1. Базовая RAG-система: поиск, сборка контекста, ответ по схеме и оценка

2.6. Метрики

Качество поиска можно оценить только там, где известно, какие документы нужно было найти. Это утверждения с метками `SUPPORT` и `CONTRADICT`: для каждого из них в разметке указаны документы-свидетельства. Для утверждений `NOINFO` искать нечего, поэтому в метрики поиска они не входят. Множество утверждений, на которых оценивается поиск, обозначим Q^+; в отладочной части (dev, п. 3.1) это 60 утверждений из 90.

Метрики поиска учитывают только первые k документов выдачи — отсюда «@k» в названиях. Для утверждения q из Q^+ пусть $\mathrm{Rel}(q)$ — множество его релевантных документов из разметки, $R_k(q)$ — первые k найденных документов, r_q — ранг первого релевантного документа в выдаче:

$$\mathrm{Hit@}k = \mathbf{1}[\mathrm{Rel}(q) \cap R_k(q) \neq \emptyset], \qquad \mathrm{Recall@}k = \frac{|\mathrm{Rel}(q) \cap R_k(q)|}{|\mathrm{Rel}(q)|}, \qquad \mathrm{RR@}k = \begin{cases} 1/r_q, & r_q \leq k, \\ 0, & \text{иначе.} \end{cases}$$

Что показывает каждая метрика:

- **Hit@k** (попадание) — есть ли среди первых k документов хотя бы один релевантный: 1 или 0. Если Hit@k = 0, верную ссылку система дать не может: нужного документа в контексте нет.
- **Recall@k** (полнота) — какая доля всех релевантных документов утверждения попала в первые k. От Hit@k она отличается, только когда релевантных документов несколько; в dev таких утверждений 5 из 60, поэтому здесь две метрики почти совпадают.
- **RR@k** (reciprocal rank, обратный ранг) — насколько высоко стоит первый релевантный документ: 1 на первом месте, 1/2 на втором, 1/3 на третьем и 0, если в первых k его нет. Место в выдаче важно, потому что контекст собирается по порядку ранга (п. 2.3): чем ниже документ, тем вероятнее, что его обоснование не поместится в бюджет.

Пример: у утверждения два релевантных документа, и поиск поставил их на 2-е и 6-е места. При $k = 3$ получается Hit@3 = 1 (нужный документ в выдаче есть), Recall@3 = 1/2 (найден один из двух) и RR@3 = 1/2 (первый релевантный — на втором месте). При $k = 1$ все три метрики равны нулю.

Все три усредняются по Q^+: среднее Hit@k — это доля утверждений, для которых поиск нашёл хотя бы одно свидетельство, а среднее RR@k называют MRR@k (mean reciprocal rank). Отдельно считается **доля переданного обоснования** — для скольких утверждений Q^+ обоснование целиком попало в промпт.

Ответ системы оценивается по всем утверждениям:

- **Accuracy** — доля верных меток. Неразобранный ответ и ошибка вызова считаются неверными («Правила», п. 6).
- **Полнота по классам** (recall каждого класса) — показывает, не отвечает ли модель на всё «NOINFO».
- **Верная ссылка** — для утверждений `SUPPORT` и `CONTRADICT`: метка верна **и** названные предложения содержат эталонное обоснование. Это согласие с разметкой, а не доказательство того, что модель рассуждала по этим предложениям.

3. Что сделать

3.1. Данные и разбиение

1. Скачайте официальный релиз SciFact [4] (адрес и контрольная сумма — в приложении) и запишите в манифест источник, SHA-256 архива и файлов.
2. Выведите метку каждого утверждения: пустой `evidence` — `NOINFO`; иначе — метка обоснований. Если у утверждения встретятся обе метки, исключите его и запишите причину; в `claims_dev.jsonl` таких нет.
3. Разбейте `claims_dev.jsonl` до любых опытов. В каждом классе упорядочьте утверждения по SHA-256 строки `scifact:<id>` (код в приложении) и возьмите первые 30 в отладочную часть. Получится **dev из 90 утверждений** (по 30 на класс) и **holdout из 210** (82 `NOINFO`, 94 `SUPPORT`, 34 `CONTRADICT`). Holdout в этой работе не используется: он понадобится для итоговой проверки в ЛР3. Сохраните оба списка в манифест.
4. Для проверки форматов и прогрева используйте утверждения из `claims_train.jsonl`.

Эталон (`evidence`, метки, `cited_doc_ids`) читает только модуль оценки; поиск, сборка контекста и модель его не видят («Правила», п. 5).

3.2. Индекс

Выберите один кодировщик из sentence-transformers [3, 6]: `BAAI/bge-small-en-v1.5`, `intfloat/e5-small-v2` или `sentence-transformers/all-MiniLM-L6-v2`. Следуйте карточке модели: у E5 перед текстом ставятся префиксы `query:` и `passage:`, у BGE к запросу добавляется рекомендованная инструкция. В паспорт индекса запишите модель и ревизию, размерность, предел длины, нормализацию и префиксы, а также долю обрезанных аннотаций.

Текст документа — заголовок, перевод строки и предложения аннотации через пробел; запрос — текст утверждения. Сохраните матрицу векторов, соответствие строк и `doc_id` и хеш матрицы. Для каждого утверждения сохраните первые 5 документов с оценками: варианты $k = 1, 3, 5$ — начала этого списка.

3.3. Контекст, промпт и вызов модели

- **Бюджет** — 2048 входных токенов после применения шаблона диалога; сюда входят инструкция, утверждение и документы. Считайте токены токенизатором той же модели (у vLLM и llama.cpp для этого есть метод `/tokenize`). Если токенизатор недоступен, как у части внешних API, сверяйтесь с `usage.prompt_tokens` в ответе и оставьте запас 10%.

- **Сборка**: документы по рангу; для каждого — `doc_id`, заголовок и предложения с номерами. Если очередной документ не помещается целиком, добавьте столько его предложений, сколько войдёт, и остановитесь.
- **Промпты** — по-английски, как и данные; тексты промптов сохраняются в репозитории. В RAG модель просят отвечать только по переданным документам, в LLM-only — разрешают опираться на собственные знания и выбирать `NOINFO`, если решить нельзя. Определения меток и формат ответа одинаковы во всех режимах.
- **Генерация**: температура 0, не более 128 выходных токенов, `response_format` со схемой из приложения.

Для каждого вызова в `results.jsonl` сохраняются: режим, k, найденные документы с оценками, переданные документы и номера предложений, полный промпт и число его токенов, сырой ответ, разобранный JSON, статус разбора, время поиска и генерации.

3.4. Проверки перед опытами

- Метрики поиска на известном примере: два релевантных документа, один найден на третьем месте — Hit@3 = 1, Recall@3 = 0,5, RR@3 = 1/3, а при $k = 1$ все три равны нулю.
- Проверка «содержит обоснование» ловит случай, когда из переданного документа убрано одно предложение обоснования.
- Разбор ответа отклоняет неизвестную метку, лишнее поле и ссылку на непереданное предложение.
- Вектор запроса и вектор того же текста как документа имеют близость около 1.

4. Эксперименты

До запусков запишите прогноз: на каком классе модель без поиска ошибается чаще; сколько ошибок исправит RAG и может ли он что-то испортить; как изменятся поиск и ответы при $k = 1, 3, 5$.

4.1. Эксперимент 1. Без поиска, с поиском и с эталонным свидетельством

На 90 утверждениях dev выполните по одному вызову в режимах LLM-only и RAG с $k = 3$, а в режиме oracle — на 60 утверждениях классов `SUPPORT` и `CONTRADICT`. Всего 240 вызовов.

1. Таблица: Accuracy и полнота по классам для LLM-only и RAG, доля верных ссылок для RAG.
2. Попарно LLM-only → RAG: сколько ответов RAG исправил и сколько испортил («Правила», п. 6). На 90 утверждениях один ответ — 1,1 п.п.
3. Oracle сравните с RAG и LLM-only **на тех же 60 утверждениях**, а не с общей Accuracy на 90. Если модель ошибается и с эталонным свидетельством, улучшение поиска эту ошибку не исправит.

4.2. Эксперимент 2. Сколько документов передавать

1. Для $k = 1, 3, 5$ посчитайте на Q^+ из dev Hit@k, Recall@k, MRR@k и долю переданного обоснования, а также среднее число реально переданных документов и токенов.
2. Выполните RAG при $k = 1$ и $k = 5$ на тех же 90 утверждениях (ещё 180 вызовов) и сравните Accuracy и ссылки с $k = 3$ попарно.
3. Объясните расхождения. Рост Recall@k не обязательно доходит до модели: длинный первый документ может занять весь бюджет. Лишние документы могут и мешать: модели хуже используют сведения из середины длинного контекста [7].

4.3. Разбор ошибок

Разберите четыре содержательных случая: свидетельство не найдено; найдено, но не поместилось; передано, но ответ неверен; ошибка на утверждении `NOINFO`. Для каждого покажите утверждение и метку, выдачу с оценками, переданные предложения и сырой ответ модели и назовите этап, на котором потерялась информация. Если какого-то случая нет, скажите об этом и разберите другой.

4.4. По желанию

- Отключите `response_format` и попросите JSON только в инструкции. Как изменится доля неразобранных ответов?
- Замените кодировщик. Как изменились доля обрезанных аннотаций и Recall@3?
- Переведите 20 утверждений dev на русский и повторите поиск тем же кодировщиком. Что произошло с Hit@3 и почему?

5. Что сдать

- Код, конфигурации, тексты промптов, JSON Schema ответа; манифест данных с разбиением и паспортом индекса.
- Артефакты: каталоги запусков экспериментов 1 и 2, матрица векторов и её соответствие `doc_id`.
- Отчёт на 2–3 страницы: прогноз; таблица эксперимента 1 с попарными переходами и блоком oracle; таблица по k; четыре разобранных случая; выводы; кодогенерация.

Рост Accuracy не обязателен: поиск может и не помочь. Ошибки постановки здесь чаще всего такие: эталон попал в промпт, неразобранные ответы исключены из знаменателя, скрытая обрезка промпта сервером.

Вопросы к защите

1. Модель ответила `NOINFO` на утверждение с меткой `SUPPORT`. Покажите по трём спискам, на каком этапе ей стало недоступно обоснование.
2. У утверждения два релевантных документа, один найден на третьем месте. Чему равны Hit@3, Recall@3 и RR@3, а что будет при $k = 1$?
3. Чем обрезка текста кодировщиком отличается от ограничения бюджета контекста? Где видно каждое в ваших артефактах?
4. Почему при переходе от $k = 3$ к $k = 5$ Recall вырос, а промпт у части утверждений не изменился?
5. Метка верна, а ссылка указывает на предложение без обоснования. Как такой ответ учтён в Accuracy и в доле верных ссылок?
6. Почему Accuracy oracle на 60 утверждениях нельзя сравнивать с Accuracy RAG на 90?
7. Уменьшите бюджет до 1024 токенов. Предскажите, как изменится доля переданного обоснования при $k = 3$, и пересчитайте её без новых вызовов модели.

Источники

1. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. Статья — исходная постановка RAG (разд. 1–2).
2. Wadden D. et al. Fact or Fiction: Verifying Scientific Claims. EMNLP, 2020. Статья — набор SciFact, разметка обоснований и официальная оценка (разд. 3–4).
3. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP, 2019. Статья — как устроены кодировщики предложений (разд. 3); практическое введение: семантический поиск в Sentence Transformers.
4. Allen Institute for AI. SciFact: репозиторий, описание формата данных.
5. vLLM. Structured Outputs — параметр `response_format` с JSON Schema; тот же параметр принимают llama.cpp и Ollama.
6. Карточки кодировщиков: BGE-small-en-v1.5, E5-small-v2, all-MiniLM-L6-v2.
7. Liu N. F. et al. Lost in the Middle: How Language Models Use Long Contexts. TACL, 2024. Статья — модели хуже используют сведения из середины длинного контекста (разд. 2–3).
8. Mistral AI. RAG Quickstart — базовый RAG с нуля: нарезка документа, эмбеддинги, векторная база, поиск и генерация.

9. Практический курс по LLM, модуль 4: RAG и как его готовить (ноутбук на русском) — загрузчики документов, разбиение текста, эмбеддинги, векторные хранилища и ретриверы на LangChain.
10. Bourke D. Simple Local RAG — локальный RAG над PDF с нуля: разбиение на фрагменты, эмбеддинги sentence-transformers, поиск по близости и генерация локальной моделью.

Приложение. Детали реализации

Данные. Релиз SciFact: `https://scifact.s3-us-west-2.amazonaws.com/release/latest/data.tar.gz` (3,1 МБ; SHA-256 архива на 25.09.2026 — `11c621288d41ac144d29b13b0f8503b3820b7d6e8b1f6ff24dff335c196d76be`). Нужны файлы `corpus.jsonl`, `claims_train.jsonl`, `claims_dev.jsonl`. В архиве есть служебные файлы macOS с именами на `._` — их пропускайте.

Метка и разбиение:

```
import hashlib

def claim_label(claim):
    labels = {r["label"] for rs in claim["evidence"].values() for r in rs}
    if not claim["evidence"]:
        return "NOINFO"
    return labels.pop() if len(labels) == 1 else None   # None — исключить

def split_dev(claims, per_class=30):
    by_class = {}
    for c in claims:
        by_class.setdefault(claim_label(c), []).append(c["id"])
    dev, holdout = [], []
    for label in ("SUPPORT", "CONTRADICT", "NOINFO"):
        ids = sorted(by_class[label], key=lambda i:
 ↪  hashlib.sha256(f"scifact:{i}".encode()).hexdigest())
        dev += ids[:per_class]
        holdout += ids[per_class:]
    return dev, holdout
```

Схема ответа для `response_format`:

```
{"type": "json_schema",
 "json_schema": {"name": "verdict", "strict": true,
  "schema": {"type": "object", "additionalProperties": false,
   "required": ["label", "doc_id", "sentences"],
   "properties": {
     "label": {"enum": ["SUPPORT", "CONTRADICT", "NOINFO"]},
     "doc_id": {"anyOf": [{"type": "string"}, {"type": "null"}]},
     "sentences": {"type": "array", "items": {"type": "integer"}}}}}}
```

Неотрицательность номеров, принадлежность документа и предложений к переданным проверяет ваш разбор ответа, а не схема.

Обоснование передано, если хотя бы один список `sentences` из эталона для этого документа целиком входит в множество переданных номеров предложений. **Ссылка верна**, если метка совпала с эталонной и множество предложений ответа включает хотя бы одно эталонное обоснование того же документа.

Ориентиры для самопроверки. Кодирование корпуса маленьким кодировщиком на CPU занимает несколько минут, поиск по матрице — миллисекунды на запрос. На T4 с моделью на 3–4 млрд параметров один вызов в этой работе длится 1–3 секунды, а обе серии — 10–20 минут.